

Ch 07: List Methods and Processing Lists

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

List Methods

List Methods

Lists are container objects that have numerous methods that allow you to work with the elements that they contain. Python also provides some built-in functions that are useful for working with lists.

Method	Description
<code>append(item)</code>	Adds <code>item</code> to the end of the list.
<code>index(item)</code>	Returns the index of the first element whose value is equal to <code>item</code> .
<code>insert(index,item)</code>	Inserts <code>item</code> into the list at the specified <code>index</code> . Expands list size.
<code>sort()</code>	Sorts the items in the list so they appear in ascending order.
<code>remove(item)</code>	Removes the first occurrence of <code>item</code> from the list.
<code>reverse()</code>	Reverses the order of the items in the list.



Useful Built-in Functions for Lists

- `del` statement: removes an element from a specific index in a list
 - General format: `del list[i]`
 - **Note:** this is not a function, but a keyword, it can be used for dicts and other places.
- `min()` and `max()` functions: built-in functions (not list functions)
 - return the item that has the lowest or highest value in a sequence
 - The sequence is passed as an argument



Copying Lists

Copying Lists

To make a copy of a list, you **must** copy the list's elements

Note: Assigning one list to another just makes a reference, e.g.

```
l1 = list(range(1,5))
```

```
l2 = l1
```

```
l1[0] = 42
```

```
print(l2)
```

may or may not do what you expect, depending on what you think the assignment of the lists is doing here.

- Two (+) methods to do this
 - 1 Create a new empty list and use a for loop to add a copy of each element from the original list to the new list
 - 2 Create a new empty list and concatenate the old list to the new empty list
 - 3 The slicing operator makes a copy of the sliced elements, so `new_list = old_list[:]` is often used



Processing Lists

- List elements can be used in calculations
- To calculate total of numeric values in a list use loop with accumulator variable
- To average numeric values in a list:
 - Calculate the total using running sum / accumulator
 - Divide total of the values by `len(list)`
- Lists can be passed as an argument to a function
 - **Question** do you expect the list to be a copy or a reference when passed into a function?
- A function can return a reference to a list
- To save the contents of a list to a file:
 - Use the file object's `writelines()` method
 - Does not automatically write `\n` at the end of each item
 - Use a `for` loop to write each element and `\n`
- To read data from a file use the file objects `readlines()` method



List Comprehensions

List Comprehension

A list comprehension is a concise expression that creates a new list by iterating over the elements of an existing list.

Using a for loop

```
l1 = [1, 2, 3, 4]
l2 = []
for item in l1:
    l2.append(item)
```

Using a list comprehension

```
l1 = [1, 2, 3, 4]
l2 = [item for item in l1]
```



Two-Dimensional Lists

Two-Dimensional List

A two-dimensional list is a list that has other lists as its elements.

- To process data in a two-dimensional list, need to use two index
 - Typically use nested loops to process

```
values = [[1, 2, 3],  
          [10, 20, 30],  
          [100, 200, 300]]
```

```
for row in values:  
    for element in row:  
        print(element)
```



Summary

- Lists are container objects that have numerous methods that allow you to work with the elements that they contain
 - Can `sort()`, `append()` lists, can `remove(item)` to remove an item, etc.
- Be careful when copying containers like lists, assigning one list to another only makes a reference, need to create a new list and copy elements.
- You can process lists using a `for` iterator for a count / indexed controlled loop.



References



EAST TEXAS A&M